



Indian Institute of Technology Hyderabad

Department of Computer Science & Engineering

CS3523 – Operating Systems II

End Semester Examination

Instructor: Dr. Abhijith Das

Question 1. Schedules Anyone (Matrix Disk)

Disk scheduling can become quite interesting, especially as the disk queue gets larger. We explore disk scheduling for a simplified **matrix disk**.

The matrix disk organises data into rows and columns:

[0]	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
...							

The columns all move to the left every R time steps (with wrap-around). The disk reads or writes a block in R time steps. Seeking to another row takes S time units.

Assume the following starting position for each sub-question:

[0]	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23

- (a) Assume $R = 1$ and $S = 1$. What is the **minimum time** it will take to read block 21?

Answer: Block 21 is in row 2, column 5.

From starting position (row 0):

- Seek to row 2: takes $2S = 2$ time units. During this time the columns shift by 2, so block 21 is now in column $5 - 2 = 3$.
- Wait for block 21 to come under the head: since it is already at column 3 (the head is at column 0 after shifting), wait 3 more time units.
- Read: 1 time unit.

Total = 2 seek + 3 wait + 1 read = 6 time units.

- (b) Assume $R = 2$ and $S = 1$. How long will it take to read blocks 6 and 18, if we **read 6 before 18**?

Answer: Reading block 6 first (row 0, col 6): wait 6 time units, read 2 time units. After $6 + 2 = 8$ time steps, columns have shifted by $8/2 = 4$ positions. Block 18 is in row 2, col $2 - 4 \equiv 6 \pmod{8}$.

Seek from row 0 to row 2: $2S = 2$ time units (columns shift 1 more position $\Rightarrow$ col 5).

Wait for col 5 to arrive: 5 time units. Read: 2 time units.

Total ≈ 12 wait + 2 read + 2 seek = 22 time units.

- (c) Assume $R = 2$ and $S = 1$. How long will it take to read blocks 6 and 18, if we **read 18 before 6**?

Answer: Seek to row 2 first: $2S = 2$ time units; columns shift 1 position. Block 18 now at col 1.

Wait 1 time unit, read 2 time units.

Total so far: 5 time units; columns shifted $5/2 \approx 2$ more during seek+wait+read.

Seek back to row 0: $2S = 2$ time units. Block 6 now reachable with a short wait.

Total ≈ 2 seek + 2 wait + 2 read + 2 seek + small wait = 14 time units.

- (d) Assuming $R = 1$, for what values of S should the scheduler read 18 **before** 6?

Answer: Reading 18 before 6 is beneficial when the extra seek cost is offset by reduced waiting.

Comparing the two orderings shows reading 18 first is better when:

$$S \leq 1$$

i.e. when seek time is at most 1 time unit.

- (e) Write the `how_long` function for a **shortest-seek-time-first (SSTF)** scheduler:

Answer:

```
int how_long(block, curr_row, S) {
    dest_row = block / 8;           // destination row
    seek_dist = abs(dest_row - curr_row); // distance in rows
    seek_time = seek_dist * S;      // seek time
    return seek_time;
}
```

Question 2. Concurrency

Given a basic spin lock where locking/unlocking each takes A time units, a context switch takes C time units, and a time slice is T time units long. Consider the following code executed by **two threads** on one CPU:

```
mutex_lock();
do_something();    // takes no time
mutex_unlock();
exit();            // takes no time
```

- What is the best-case time for two threads to finish?

Answer: Best case: Thread 1 runs to completion before Thread 2 starts (one context switch).

T1 runs: lock (A) + unlock (A) = 2A, then context switch (C), then T2 runs 2A.

Best-case total = $4A + C$

- What is the worst-case time (at most three context switches)?

Answer: Worst case: T1 acquires lock (A), context switch (C) to T2. T2 tries to lock and spins for a full time slice (T), context switch (C) back to T1. T1 unlocks (A), context switch (C) to T2, T2 locks (A) and unlocks (A).

Worst-case total = $4A + 3C + T$

- If the spin lock is replaced with a queue-based lock, how does the worst-case change?

Answer: A queue-based lock puts the waiting thread to sleep instead of spinning. This eliminates the wasted spin time T.

New worst-case = $4A + 3C$ (avoids the T spinning term).

Question 3. RAIDs

We examine the time to perform 12 writes to random locations within a RAID, spread evenly across disks. Each read/write takes D time units (simple model).

- 4-disk RAID-0 (striping). How long to complete 12 writes? **Answer:** 12 writes spread evenly across 4 disks = 3 writes per disk, done in parallel.
 $\approx 3D$
- 4-disk RAID-1 (mirroring). How long? **Answer:** Each write goes to 2 disks (mirrored). 12 writes $\times$ 2 copies / 4 disks = 6 writes per disk.
 $\approx 6D$
- 4-disk RAID-4 (parity). How long? **Answer:** Each write requires a read-modify-write of the parity disk: 2 reads + 2 writes per request = 4 I/Os per write. Parity disk is a bottleneck: $12 \times 2/1 = 24$ operations on parity disk.
 $\approx 24D$
- 4-disk RAID-5 (rotated parity). How long? **Answer:** Parity is distributed, removing the bottleneck. Each write still needs 4 I/Os, spread across 4 disks: $12 \times$

$4/4 = 12$ operations per disk.
 $\approx 12D$

Now assume a better disk model: random seek = S time, full rotation = R time, transfer is free.

- **RAID-0, 12 random writes:** **Answer:** *3 writes per disk; each write costs $S + \frac{R}{2}$ on average.*
 $\approx 3(S + \frac{R}{2})$
- **RAID-1, 12 random writes:** **Answer:** *6 writes per disk (worst case 2 seeks and 2 rotations).*
 $\approx 6(S + \frac{R}{2})$ (slightly longer due to possible 2 seeks per write on mirror)
- **RAID-4, 12 random writes:** **Answer:** *Each write = 2 reads + 2 writes on parity disk, costing $2(S + \frac{R}{2}) + 2(S + \frac{R}{2}) = 4(S + \frac{R}{2})$ per req; 12 requests serialised on parity disk.*
 $\approx 12(S + \frac{3R}{2})$
- **RAID-5, 12 random writes:** **Answer:** *Parity distributed; 12 writes $\times$ 4 I/Os / 4 disks = 12 I/Os per disk.*
 $\approx 6(S + \frac{3R}{2})$

Question 4. Producers and Consumers

Consider the following producer/consumer code using `Pthread_cond_broadcast`, which wakes up *all* threads waiting on a condition variable:

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        while (count == MAX)
            Pthread_cond_wait(&cv, &mutex);
        put(i);
        Pthread_cond_broadcast(&cv); // wake all!
        Pthread_mutex_unlock(&mutex);
    }
}

void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        while (count == 0)
            Pthread_cond_wait(&cv, &mutex);
        int tmp = get();
        Pthread_cond_broadcast(&cv); // wake all!
    }
}
```

```

        Pthread_mutex_unlock(&mutex);
        printf("%d\n", tmp);
    }
}

```

Does this code work correctly?

Answer: *Yes, this code is correct, though potentially inefficient.*

*The code uses only one condition variable (**cv**) for both producers and consumers. Normally this would be a bug: a consumer signalling could wake another consumer (which finds the buffer empty and goes back to sleep) instead of a producer.*

*However, using **Pthread_cond_broadcast** instead of **Pthread_cond_signal** wakes all waiting threads. Because each thread rechecks the condition in a **while** loop, threads that cannot make progress simply go back to sleep. Only those threads for which the condition is satisfied will proceed.*

Correctness: *guaranteed – no missed wakeups, no spurious forward progress.*

Inefficiency: *threads are woken unnecessarily. This is why two separate condition variables with **signal** (not **broadcast**) is the preferred idiom.*

Question 5. File System – Maximum File Size

Write the most efficient program to determine the maximum file size on the file system you are running, using only: `open()`, `read()`, `write()`, `close()`, `lseek()`.

Answer: *The most efficient approach is to write data in large chunks until the write fails (no need for binary search using **lseek**):*

```

// assume the file system has room for a big file
int fd = open("file", O_RDWR | O_CREAT | O_TRUNC);
assert(fd > 0);
char buf[4096];
int i = 0;
while (1) {
    int rc = write(fd, buf, 4096);
    if (rc <= 0)
        break;    // write failed -> hit max size
    i++;
}
printf("max_file_is_%d_bytes\n", i * 4096);
close(fd);

```

Explanation: *Writing 4096 bytes (one block) at a time is efficient because it matches a typical file-system block size, minimising overhead. The loop continues until **write()** returns ≤ 0 , indicating the file-system limit has been reached. The maximum file size is then $i \times 4096$ bytes.*